Infranex BT

Architecture Documentation

v2.0 - 3-Pillar Scoring Model

Bittensor Intelligence & Mining Operations Platform

Last updated: 2026-08-29

Private - Internal Operations Platform

Table of Contents

1. System Architecture

2. v2.0 Scoring Engine - 3-Pillar Model

3. Decision Engine - RUN / WATCH / AVOID

4. Scoring API Reference

5. Migration Guide

6. Module Inventory

7. Technology Stack

8. Security Principles

1. System Architecture

Overview

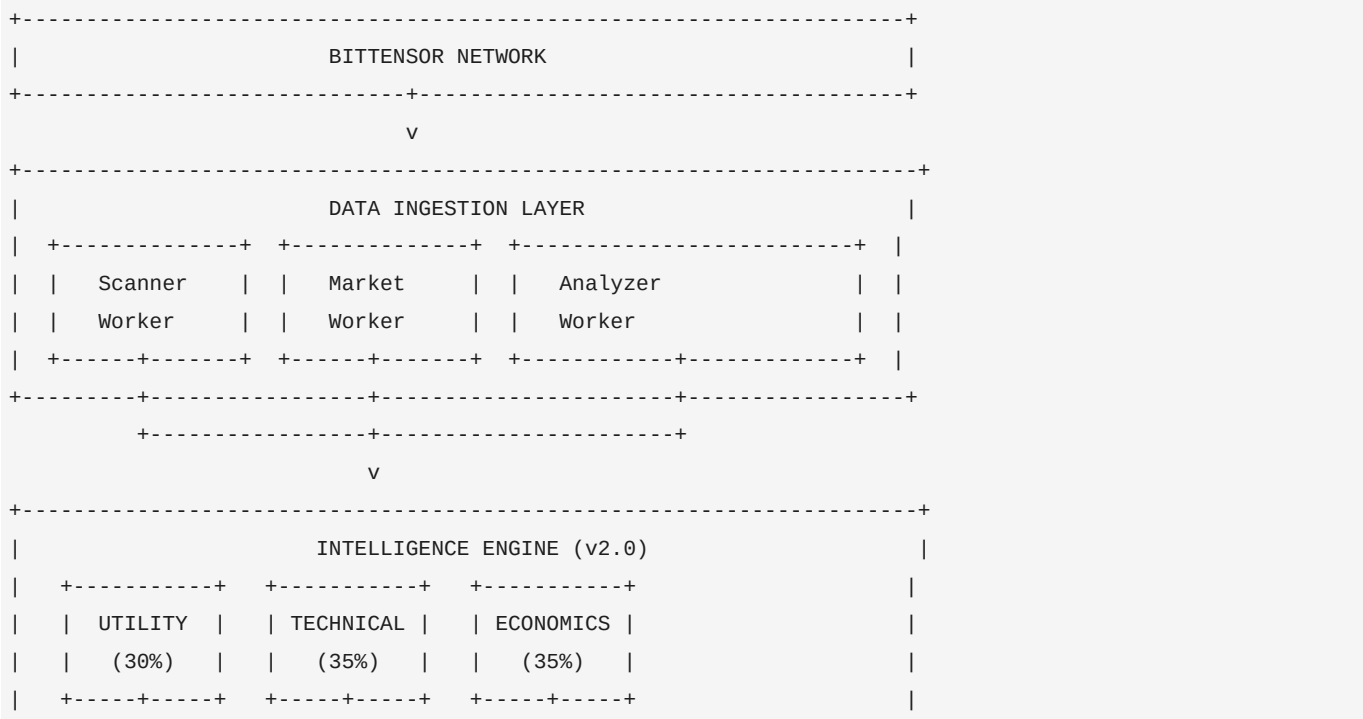
Infranex BT is a production-ready platform for discovering, analyzing, and deploying Bittensor mining operations with GPU infrastructure automation.

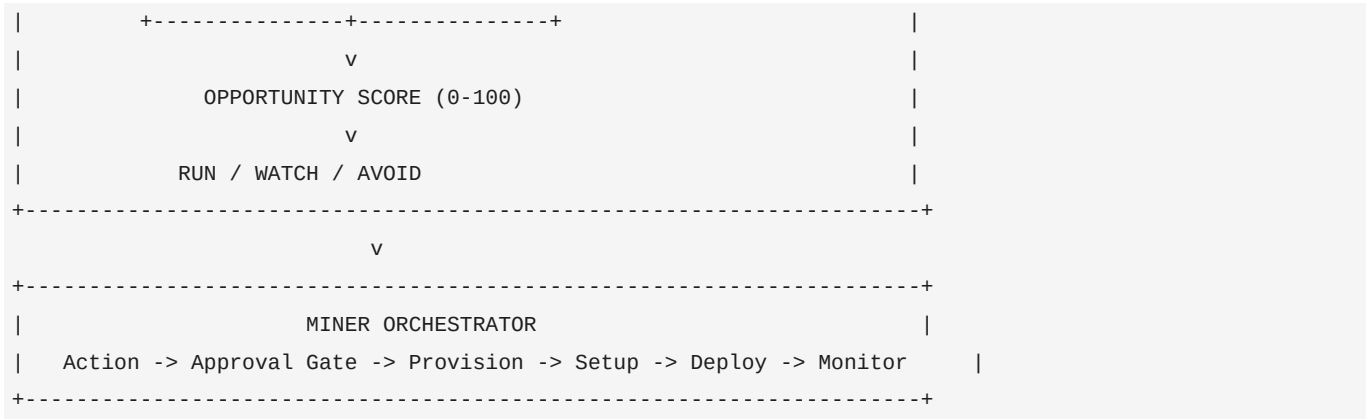
Data Flow

The system follows a layered architecture:

- Data Ingestion Layer - Scanner, Market, and Analyzer workers collect data from Bittensor, CoinGecko, and GitHub
- Intelligence Engine - 3-pillar scoring model computes opportunity scores
- Strategy Engine - Converts scores into RUN/WATCH/AVOID decisions
- Miner Orchestrator - Manages the full miner lifecycle
- DevOps Engine - Provider APIs, deployment, health checks
- Monitoring & Learning - Real-time alerts and feedback loop

Architecture Diagram




OPPORTUNITY SCORE (0-100)
RUN / WATCH / AVOID
MINER ORCHESTRATOR
Action -> Approval Gate -> Provision -> Setup -> Deploy -> Monitor

2. v2.0 Scoring Engine - 3-Pillar Model

Overview

The v2.0 scoring model reorganizes subnet opportunity assessment into three intuitive pillars. Each pillar produces a sub-score (0-100). The weighted combination produces the Opportunity Score.

Pillar 1: Utility (30%)

Question: What does the subnet actually do? Is it a useful service?

Factor	Weight	Source
Problem clarity	30%	README analysis
Uniqueness	20%	Subnet metadata
Description quality	15%	Subnet description
Development activity	15%	GitHub commits
Tokenomics	10%	Subnet registration
Governance	10%	Subnet metadata

Pillar 2: Technical (35%)

Question: Can we actually run this subnet? What hardware do we need?

Factor	Weight	Source
GPU match rate	40%	SubnetRequirement + GPUModel
Dependency simplicity	20%	SubnetRequirement.dependencies
CUDA compatibility	15%	SubnetRequirement.cuda_version
Resource feasibility	15%	RAM, storage requirements
Data confidence	10%	extraction_confidence

Pillar 3: Economics (35%)

Question: Will this subnet be profitable?

Factor	Weight	Source
Revenue potential	30%	Emission * incentive * TAO price
Competition health	20%	top_5_concentration
Reward stability	15%	top vs median incentive spread
Cost efficiency	20%	GPU cost vs expected revenue
Market momentum	15%	Price change, liquidity

Opportunity Score Computation

```
total = (utility * 0.30) + (technical * 0.35) + (economics * 0.35)
decision = RUN if total >= 75 else WATCH if total >= 40 else AVOID
```

3. Decision Engine - RUN / WATCH / AVOID

Decision Bands

Score	Decision	Action
≥ 75	RUN	Deploy miner on this subnet
40-74	WATCH	Monitor for improvement
< 40	AVOID	Do not deploy; exit if running

Portfolio Constraints

Before emitting a RUN decision, the engine checks portfolio constraints:

- Max concurrent miners (default: 10)
- Max monthly spend (default: ₹500,000)
- Max spend per miner (default: ₹50,000)
- Min days between entries (default: 7)
- Allowed subnet categories

Constraint Violation Handling

Violation	Effect
Max miners reached	Downgrade RUN -> WATCH
Monthly spend exceeded	Downgrade RUN -> WATCH
Per-miner spend too high	Downgrade RUN -> WATCH
Too soon since last entry	Downgrade RUN -> WATCH
Category not allowed	Block deployment

Decision Flow

```
1. Compute Opportunity Score
   +-- Utility Score (30%)
   +-- Technical Score (35%)
   +-- Economics Score (35%)
```

```
2. Apply Decision Thresholds
   +-- score >= 75 -> candidate: RUN
   +-- score >= 40 -> candidate: WATCH
   +-- score < 40 -> candidate: AVOID

3. Check Portfolio Constraints
   +-- If RUN and constraints violated -> downgrade to WATCH
   +-- If RUN and constraints OK -> emit RUN
   +-- WATCH/AVOID -> emit as-is

4. Apply Approval Gates
   +-- RUN -> L2_confirm (needs human approval)
   +-- WATCH -> L1_auto (logged, no approval)
   +-- AVOID -> L1_auto (logged, no approval)

5. Emit Action -> consumed by Orchestrator
```

4. Scoring API Reference

Compute Opportunity Score (v2.0)

```
POST /api/v2/opportunities/score

Request:
{
  "netuid": 1,
  "include_components": true
}

Response:
{
  "netuid": 1,
  "subnet_name": "text-prompting",
  "total_score": 72.5,
  "model_version": "v2.0",
  "decision": "WATCH",
  "pillar_scores": {
    "utility": 68.0,
    "technical": 82.0,
    "economics": 65.0
  },
  "components": [...]
}
```

Get Decision

```
GET /api/opportunities/{netuid}/decision

Response:
{
  "netuid": 1,
  "decision": "WATCH",
  "score": 72.5,
  "pillar_scores": { ... },
  "constraints_violated": [],
  "approval_level": "L1_auto"
}
```

List All Decisions

```
GET /api/opportunities/decisions
GET /api/opportunities/decisions?decision=RUN
GET /api/opportunities/decisions?min_score=75
```

Trigger Scoring Run

```
POST /api/opportunities/score-all

Response:
{
  "status": "scoring_started",
  "netuids": [1, 3, 64],
  "estimated_seconds": 30
}
```

5. Migration Guide

v1.0 -> v2.0 Mapping

v1.0	v2.0	Notes
8 components	3 pillars	Simpler, more intuitive
ENTER	RUN	Same threshold (≥75)
HOLD	WATCH	Same band (40-74)
EXIT	AVOID	Same threshold (<40)

Database Migration

```
ALTER TABLE opportunity_scores
  ADD COLUMN utility_score FLOAT,
  ADD COLUMN technical_score FLOAT,
  ADD COLUMN economics_score FLOAT,
  ADD COLUMN pillar_version VARCHAR(10) DEFAULT 'v2.0',
  ADD COLUMN decision VARCHAR(10);

CREATE TABLE utility_assessments (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  netuid INTEGER,
  problem_clarity FLOAT,
  uniqueness FLOAT,
  description_quality FLOAT,
  development_activity FLOAT,
  tokenomics_score FLOAT,
  governance_score FLOAT,
  overall_utility FLOAT,
  assessed_at TIMESTAMPTZ DEFAULT NOW()
);
```

Implementation Order

- 1. Database migration - Add new columns and table
- 2. Scoring engine - Implement 3-pillar scoring
- 3. API routes - Add v2 endpoints
- 4. Strategy engine - Update decision labels

- 5. Opportunity service - Persist pillar scores
- 6. Learning engine - Track pillar accuracy
- 7. Frontend types - Update TypeScript interfaces
- 8. Frontend components - Add pillar visualization
- 9. Tests - Write tests for new scoring
- 10. Documentation - Update API docs

6. Module Inventory

Module	Location	Purpose
Intelligence Engine	backend/app/intelligence/	Scoring, GPU matching, profitability, monitoring
Strategy Engine	backend/app/strategy/	Score -> Action decision layer
Miner Orchestrator	backend/app/orchestrator/	Lifecycle state machine
Optimizer	backend/app/optimizer/	Cost savings, subnet alternatives
Recovery	backend/app/recovery/	Health checks, recovery strategies
Learning Engine	backend/app/learning/	Feedback loop, weight adaptation
Compatibility Lab	backend/app/compatibility/	GPU/server requirement validation
Monitoring	backend/app/monitoring/	Real-time alerts, SSE
Providers	backend/app/providers/	GPU provider adapters
Deployment	backend/app/deployment/	Deploy lifecycle, health checks
Approval	backend/app/approval/	L1/L2/L3 approval gates

7. Technology Stack

Layer	Technology
Frontend	Next.js 15, React 19, TypeScript, Tailwind CSS, shadcn/ui, Recharts
Backend	FastAPI, Python 3.11+, Pydantic v2, SQLAlchemy, Alembic
Database	Supabase PostgreSQL (RLS, migrations, realtime)
Auth	Supabase Auth (JWT, OAuth)
Cache	Redis/Valkey
Workers	Python async workers, triggered by Vercel Cron
Containerization	Docker multi-stage builds
CI/CD	GitHub Actions -> Vercel
Bittensor	Official Python SDK (bittensor>=9.0)

8. Security Principles

Principle	Implementation
No Coldkey Storage	Coldkeys never stored in DB, logs, env, or frontend
Hotkey Handling	Hotkeys provided at deployment time only, encrypted in transit
Approval Gates	Every deployment requires explicit user confirmation
Audit Logging	All actions logged with user, timestamp, decision
RLS	Row-level security on all user/data tables
No Auto-Spend	Zero automatic financial transactions

Approval Levels

Level	Meaning	Example Actions
L1_auto	Safe, reversible, no money. Executes immediately.	Refresh metrics, fetch status, score opportunities
L2_confirm	Small financial impact OR reversible operational change.	Rebalance under cap, restart miner, recover container
L3_mandatory	Capital-intensive, irreversible, or strategy-shifting.	Deploy new miner, migrate subnet, increase spend cap